

Java: 1. Clases y Objetos

```
// Definición de una clase
public class Usuario {
    // Atributos (características)
    private String nombre;
    private int edad;

    // Constructor (método especial para crear objetos)
    public Usuario(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    // Método (comportamiento)
    public void saludar() {
        System.out.println("Hola, soy " + nombre);
    }
}
```

```
// Crear objetos (instancias)
Usuario usuario1 = new Usuario("Ana", 25);
Usuario usuario2 = new Usuario("Carlos", 30);
usuario1.saludar(); // Imprime: Hola, soy Ana
```

2. Modificadores de acceso

```
public class EjemploAcceso {  
    // public: Accesible desde cualquier parte  
    public String nombrePublico = "Visible para todos";  
  
    // private: Solo accesible dentro de esta clase  
    private String nombrePrivado = "Solo visible aquí";  
  
    // protected: Accesible en el mismo paquete y subclases  
    protected String nombreProtegido = "Visible en herencia";  
  
    // default (sin modificador): Solo en el mismo paquete  
    String nombreDefault = "Visible en el paquete";  
  
    public void metodoPublico() {  
        System.out.println(nombrePrivado); // ✅ Puedo acceder aquí  
    }  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        EjemploAcceso ej = new EjemploAcceso();  
        System.out.println(ej.nombrePublico); // ✅ OK  
        // System.out.println(ej.nombrePrivado); // ❌ Error!  
    }  
}
```

3. Encapsulamiento Getters y Setters

Proteger los datos y controlar cómo se accede a ellos.

```
public class CuentaBancaria {  
    private double saldo; // Atributo privado  
  
    // Getter - permite ver el saldo de forma controlada  
    public double getSaldo() {  
        return saldo;  
    }  
  
    // Setter - permite modificar el saldo con validaciones  
    public void depositar(double monto) {  
        if (monto > 0) {  
            this.saldo += monto;  
        } else {  
            throw new IllegalArgumentException("Monto debe ser positivo");  
        }  
    }  
}
```

```
// Método de negocio, no solo un setter  
public boolean retirar(double monto) {  
    if (monto > 0 && monto <= saldo) {  
        this.saldo -= monto;  
        return true;  
    }  
    return false;  
}
```

4. Herencia

Una clase puede heredar atributos y métodos de otra clase.

```
// Clase padre (superclase)
public class Animal {
    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public void hacerSonido() {
        System.out.println("El animal hace un sonido");
    }
}
```

```
// Clase hija (subclase)
public class Perro extends Animal {
    private String raza;

    public Perro(String nombre, String raza) {
        super(nombre); // Llama al constructor del padre
        this.raza = raza;
    }

    // Sobrescritura de método (override)
    @Override
    public void hacerSonido() {
        System.out.println(nombre + " dice: Guau guau!");
    }

    // Método específico de Perro
    public void moverCola() {
        System.out.println(nombre + " mueve la cola");
    }
}
```

```
// Uso
```

```
Animal animal = new Animal("Genérico");
```

```
Perro perro = new Perro("Max", "Labrador");
```

```
animal.hacerSonido(); // "El animal hace un sonido"
```

```
perro.hacerSonido(); // "Max dice: Guau guau!"
```

```
perro.moverCola(); // "Max mueve la cola"
```

5. Interfaces

```
// Definición de interfaz
public interface Reproducible {
    void reproducir();
    void pausar();
    void detener();

    // Método default (Java 8+)
    default void siguiente() {
        System.out.println("Pasando al siguiente");
    }
}
```

```
// Implementación de la interfaz
public class ReproductorMusica implements Reproducible {
    private String cancionActual;

    @Override
    public void reproducir() {
        System.out.println("Reproduciendo: " + cancionActual);
    }

    @Override
    public void pausar() {
        System.out.println("Música pausada");
    }

    @Override
    public void detener() {
        System.out.println("Música detenida");
        cancionActual = null;
    }
}
```

```
// Otra implementación
public class ReproductorVideo implements Reproducible {
    @Override
    public void reproducir() {
        System.out.println("Reproduciendo video");
    }

    @Override
    public void pausar() {
        System.out.println("Video pausado");
    }

    @Override
    public void detener() {
        System.out.println("Video detenido");
    }
}
```

```
// Uso polimórfico
```

```
Reproducible reproductor = new ReproductorMusica();
```

```
reproductor.reproducir();
```

```
reproductor = new ReproductorVideo();
```

```
reproductor.reproducir(); // Cambia el comportamiento
```

6. Polimorfismo

Capacidad de un objeto para tomar múltiples formas.

```
public interface Pago {  
    void procesar(double monto);  
}  
  
public class PagoTarjeta implements Pago {  
    @Override  
    public void procesar(double monto) {  
        System.out.println("Procesando pago con tarjeta: $" + monto);  
    }  
}  
  
public class PagoEfectivo implements Pago {  
    @Override  
    public void procesar(double monto) {  
        System.out.println("Procesando pago en efectivo: $" + monto);  
    }  
}
```

```
public class PagoEfectivo implements Pago {  
    @Override  
    public void procesar(double monto) {  
        System.out.println("Procesando pago en efectivo: $" + monto);  
    }  
}  
  
public class ProcesadorPagos {  
    public void ejecutarPago(Pago pago, double monto) {  
        // No necesita saber qué tipo de pago es  
        pago.procesar(monto);  
    }  
}  
  
// Uso  
ProcesadorPagos procesador = new ProcesadorPagos();  
procesador.ejecutarPago(new PagoTarjeta(), 100.50);  
procesador.ejecutarPago(new PagoEfectivo(), 50.00);
```

7. Anotaciones

Metadatos que proporcionan información sobre el código.

```
// Anotación personalizada
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Auditable {
    String accion() default "CONSULTA";
    boolean registrar() default true;
}
```

```
// Uso de anotaciones
@Service
public class UsuarioService {

    @Autowired // Anotación de Spring - inyecta dependencia
    private UserRepository repository;

    @Auditable(accion = "CREAR") // Anotación personalizada
    @Transactional // Anotación de Spring
    public Usuario crearUsuario(UsuarioDto dto) {
        return repository.save(dto.toEntity());
    }

    @Override // Anotación de Java - indica sobrescritura
    public String toString() {
        return "UsuarioService";
    }
}
```

8. Tipos Genéricos

Permiten crear clases y métodos que trabajan con tipos de forma segura.

```
// Clase genérica
public class Caja<T> {
    private T contenido;

    public void guardar(T contenido) {
        this.contenido = contenido;
    }

    public T obtener() {
        return contenido;
    }
}
```

```
// Uso
Caja<String> cajaDeTexto = new Caja<>();
cajaDeTexto.guardar("Hola Mundo");
String texto = cajaDeTexto.obtener(); // No necesita casting

Caja<Integer> cajaDeNumero = new Caja<>();
cajaDeNumero.guardar(42);
Integer numero = cajaDeNumero.obtener();

// En Spring - muy común en repositorios
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    // JpaRepository<Entidad, TipoDeId>
    List<Usuario> findByNombre(String nombre);
}
```

9. Optional

Contenedor que puede o no contener un valor (evita NullPointerException).

```
@Service
public class UsuarioService {
    @Autowired
    private UsuarioRepository repository;

    // Sin Optional - propenso a errores
    public Usuario buscarPorIdViejo(Long id) {
        Usuario usuario = repository.findById(id).orElse(null);
        if (usuario == null) {
            throw new RuntimeException("No encontrado");
        }
        return usuario;
    }
}
```

```
// Sin Optional - propenso a errores
public Usuario buscarPorIdViejo(Long id) {
    Usuario usuario = repository.findById(id).orElse(null);
    if (usuario == null) {
        throw new RuntimeException("No encontrado");
    }
    return usuario;
}

// Con Optional - más elegante y seguro
public Usuario buscarPorId(Long id) {
    return repository.findById(id)
        .orElseThrow(() -> new UsuarioNoEncontradoException("ID: " + id));
}

public String obtenerEmailUsuario(Long id) {
    return repository.findById(id)
        .map(Usuario::getEmail) // Solo si existe el usuario
        .orElse("Email no disponible");
}
}
```



10. Streams y Lambdas (Java 8+)

```
public class EjemploStreams {
    public static void main(String[] args) {
        List<Usuario> usuarios = Arrays.asList(
            new Usuario("Ana", 25, "ana@email.com"),
            new Usuario("Carlos", 17, "carlos@email.com"),
            new Usuario("Maria", 30, "maria@email.com"),
            new Usuario("Juan", 15, "juan@email.com")
        );

        // Filtrar usuarios mayores de edad y obtener sus emails
        List<String> emailsMayores = usuarios.stream()
            .filter(u -> u.getEdad() >= 18) // Filtra
            .map(Usuario::getEmail)         // Transforma
            .collect(Collectors.toList());   // Colecciona

        // Calcular edad promedio de usuarios activos
        double edadPromedio = usuarios.stream()
            .filter(Usuario::isActivo)
            .mapToInt(Usuario::getEdad)
            .average()
            .orElse(0);

        // Agrupar usuarios por ciudad
        Map<String, List<Usuario>> porCiudad = usuarios.stream()
            .collect(Collectors.groupingBy(Usuario::getCiudad));

        // Encontrar el usuario más joven
        Optional<Usuario> masJoven = usuarios.stream()
            .min(Comparator.comparing(Usuario::getEdad));
    }
}
```

```
// Excepción personalizada
```

```
public class SaldoInsuficienteException extends RuntimeException {  
    private double saldoActual;  
    private double montoRequerido;  
  
    public SaldoInsuficienteException(double saldo, double monto) {  
        super(String.format("Saldo insuficiente: $%.2f, requerido: $%.2f", saldo, monto));  
        this.saldoActual = saldo;  
        this.montoRequerido = monto;  
    }  
}
```



```
@Service
public class CuentaService {

    public void transferir(Long origenId, Long destinoId, double monto) {
        // Try-catch para manejar excepciones
        try {
            Cuenta origen = cuentaRepository.findById(origenId)
                .orElseThrow(() -> new CuentaNoEncontradaException(origenId));

            if (origen.getSaldo() < monto) {
                throw new SaldoInsuficienteException(origen.getSaldo(), monto);
            }

            origen.retirar(monto);
            // ... lógica de transferencia

        } catch (SaldoInsuficienteException e) {
            // Log específico
            log.error("Error en transferencia: {}", e.getMessage());
            throw e; // Relanzar para que el controller lo maneje
        } finally {
            // Siempre se ejecuta
            log.info("Intento de transferencia finalizado");
        }
    }
}
```

```
// Manejo global en Spring
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(SaldoInsuficienteException.class)
    public ResponseEntity<ErrorResponse> manejarSaldoInsuficiente(
        SaldoInsuficienteException e) {
        return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(new ErrorResponse(e.getMessage(), "INSUFFICIENT_FUNDS"));
    }
}
```

12. Inversión de Control (IoC) / DI

Principio fundamental en Spring - las dependencias se "inyectan" en lugar de crearlas.

```
// MAL - Acoplamiento fuerte
public class MalServicio {
    private UsuarioRepository repository;

    public MalServicio() {
        // Crea su propia dependencia - difícil de testear
        this.repository = new UsuarioRepository();
    }
}
```

```
// Bien Servicio por Controller
@Service
public class BienServicio {
    private final UsuarioRepository repository;
    private final EmailService emailService;

    // Spring inyecta las dependencias automáticamente
    public BienServicio(UsuarioRepository repo, EmailService email) {
        this.repository = repo;
        this.emailService = email;
    }

    public Usuario crearUsuario(UsuarioDto dto) {
        Usuario usuario = repository.save(dto.toEntity());
        emailService.enviarBienvenida(usuario);
        return usuario;
    }
}
```



```
// Test fácil
```

```
class BienServicioTest {
```

```
    @Test
```

```
    void testCrearUsuario() {
```

```
        UsuarioRepository mockRepo = mock(UsuarioRepository.class);
```

```
        EmailService mockEmail = mock(EmailService.class);
```

```
        BienServicio servicio = new BienServicio(mockRepo, mockEmail);
```

```
        // Test fácil con mocks
```

```
    }
```

```
}
```

Dato Primitivos

```
public class TiposPrimitivos {  
    public static void main(String[] args) {  
        // Números enteros  
        byte edadByte = 25;           // 8 bits, -128 a 127  
        short cantidadShort = 1000; // 16 bits, -32,768 a 32,767  
        int edadInt = 30;             // 32 bits, -2,147M a 2,147M (MÁS USADO)  
        long poblacionLong = 7800000000L; // 64 bits, necesita L al final  
  
        // Números decimales  
        float precioFloat = 99.99f; // 32 bits, necesita f al final  
        double precioDouble = 99.99; // 64 bits, MÁS USADO para decimales  
  
        // Otros  
        char letraChar = 'A';        // Un solo carácter  
        boolean activoBoolean = true; // true o false  
  
        // Uso típico en Spring  
        int id = 123;  
        double monto = 1500.50;  
        boolean esActivo = true;  
    }  
}
```

```
public class ClasesWrapper {  
    public static void main(String[] args) {  
        // Wrappers (con mayúscula inicial)  
        Integer edad = 25;           // Wrapper de int  
        Double precio = 99.99;       // Wrapper de double  
        Boolean activo = true;        // Wrapper de boolean  
        Long poblacion = 7800000000L; // Wrapper de long  
        Character letra = 'A';        // Wrapper de char  
  
        // Utilidades de los wrappers  
        // Convertir String a número  
        String numeroTexto = "123";  
        int numero = Integer.parseInt(numeroTexto);  
        double decimal = Double.parseDouble("45.67");  
  
        // Comparaciones seguras  
        Integer a = 100;  
        Integer b = 100;  
        System.out.println(a.equals(b)); // true (usa equals, no ==)  
  
        // Constantes útiles  
        System.out.println(Integer.MAX_VALUE); // 2147483647  
        System.out.println(Double.MIN_VALUE);   // 4.9E-324  
    }  
}
```

Array:

```
public class EjemploArrays {  
    public static void main(String[] args) {  
        // Declaración e inicialización  
        int[] numeros = new int[5];           // [0,0,0,0,0]  
        String[] nombres = new String[3];     // [null, null, null]  
  
        // Inicialización directa  
        int[] valores = {10, 20, 30, 40, 50};  
        String[] colores = {"rojo", "verde", "azul"};  
  
        // Acceso y modificación  
        valores[0] = 100;  
        System.out.println(valores[2]); // 30  
  
        // Recorrer array  
        for (int i = 0; i < valores.length; i++) {  
            System.out.println(valores[i]);  
        }  
  
        // For-each (recomendado)  
        for (int valor : valores) {  
            System.out.println(valor);  
        }  
  
        // Arrays multidimensionales  
        int[][] matriz = {  
            {1, 2, 3},  
            {4, 5, 6},  
            {7, 8, 9}  
        };  
        System.out.println(matriz[1][2]); // 6  
    }  
}
```

```
prompt.txt nuevo1 x
1  import java.util.*;
2
3  public class EjemploColecciones {
4      public static void main(String[] args) {
5
6          // LISTAS - ordenadas, permiten duplicados
7          List<String> lista = new ArrayList<>();
8          lista.add("Juan");
9          lista.add("Ana");
10         lista.add("Carlos");
11         lista.add(1, "Pedro"); // Inserta en posición 1
12
13         System.out.println(lista.get(0)); // "Juan"
14         System.out.println(lista.size()); // 4
15         System.out.println(lista.contains("Ana")); // true
16
17         // Recorrer lista
18         for (String nombre : lista) {
19             System.out.println(nombre);
20         }
21
22         // LinkedList - mejor para inserciones/eliminaciones frecuentes
23         List<String> linkedList = new LinkedList<>();
24
25         // SETS - no ordenados, sin duplicados
26         Set<String> conjunto = new HashSet<>();
27         conjunto.add("Juan");
28         conjunto.add("Ana");
29         conjunto.add("Juan"); // No se añade, ya existe
30
31         System.out.println(conjunto.size()); // 2
32         System.out.println(conjunto.contains("Ana")); // true
33     }
```

Colecciones:

```
31 System.out.println(conjunto.size()); // 2
32 System.out.println(conjunto.contains("Ana")); // true
33
34 // TreeSet - ordenado automáticamente
35 Set<Integer> numerosOrdenados = new TreeSet<>();
36 numerosOrdenados.add(5);
37 numerosOrdenados.add(1);
38 numerosOrdenados.add(3);
39 // [1, 3, 5] automáticamente ordenado
40
41 // MAPS - clave-valor
42 Map<String, Integer> edades = new HashMap<>();
43 edades.put("Juan", 25);
44 edades.put("Ana", 30);
45 edades.put("Carlos", 28);
46
47 System.out.println(edades.get("Ana")); // 30
48 System.out.println(edades.containsKey("Juan")); // true
49 System.out.println(edades.keySet()); // ["Juan", "Ana", "Carlos"]
50
51 // Recorrer Map
52 for (Map.Entry<String, Integer> entry : edades.entrySet()) {
53     System.out.println(entry.getKey() + ": " + entry.getValue());
54 }
55
56 // Map con Java 8+
57 edades.forEach((nombre, edad) ->
58     System.out.println(nombre + " tiene " + edad + " años")
59 );
60 }
61 }
```

```
1  import java.time.*;
2  import java.time.format.DateTimeFormatter;
3  import java.time.temporal.ChronoUnit;
4
5  public class EjemploFechas {
6      public static void main(String[] args) {
7
8          // Fecha actual
9          LocalDate hoy = LocalDate.now();
10         LocalDateTime ahora = LocalDateTime.now();
11
12         // Crear fechas específicas
13         LocalDate fechaNacimiento = LocalDate.of(1990, 5, 15);
14         LocalTime horaApertura = LocalTime.of(9, 30);
15
16         // Parsear fechas desde String
17         LocalDate fecha = LocalDate.parse("2023-12-25");
18         LocalDateTime fechaHora = LocalDateTime.parse("2023-12-25T10:30:00");
19
20         // Operaciones con fechas
21         LocalDate manana = hoy.plusDays(1);
22         LocalDate mesPasado = hoy.minusMonths(1);
23         LocalDate siguienteSemana = hoy.plusWeeks(1);
24
25         // Diferencia entre fechas
26         long diasEntre = ChronoUnit.DAYS.between(fechaNacimiento, hoy);
27         long añosEntre = ChronoUnit.YEARS.between(fechaNacimiento, hoy);
28
29         // Comparar fechas
30         System.out.println(hoy.isAfter(fechaNacimiento)); // true
31         System.out.println(hoy.isBefore(LocalDate.of(2025, 1, 1))); // true
```

```
29 // Comparar fechas
30 System.out.println(hoy.isAfter(fechaNacimiento)); // true
31 System.out.println(hoy.isBefore(LocalDate.of(2025, 1, 1))); // true
32
33 // Formatear fechas
34 DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
35 String fechaFormateada = hoy.format(formatter); // "26/03/2024"
36
37 DateTimeFormatter formatterCompleto =
38     DateTimeFormatter.ofPattern("dd 'de' MMMM 'de' yyyy");
39 // "26 de marzo de 2024"
40
41 // Zonas horarias
42 ZonedDateTime ahoraEnTokyo = ZonedDateTime.now(ZoneId.of("Asia/Tokyo"));
43 ZonedDateTime ahoraEnNY = ZonedDateTime.now(ZoneId.of("America/New_York"));
44 }
45 }
```

Cierre

- Generación de código con IA.
- Uso del tablero Kanban.